Processor Verification
Lab 7 - Formal Verification

Lorenzo Pattaro Zonta

May 22, 2026

1 Introduction

2 Design Under Test

The packet module implements a finite state machine (FSM) that models the reception of a packet through successive stages: header, payload, checksum, and completion. The FSM transitions through multiple states as indicated by external control signals, and generates status flags (valid_pkt, error_pkt) to indicate whether a packet was successfully received or failed due to checksum errors.

Signal	Direction	Width	Description
clk	Input	1	System clock. All state transitions occur on the rising edge.
reset	Input	1	Synchronous reset. Forces the FSM back to IDLE and clears outputs.
start_pkt	Input	1	Initiates packet reception. Valid only in IDLE.
hdr_done	Input	1	Signals completion of header processing. Valid only in HEADER.
payload_done	Input	1	Signals completion of payload processing. Valid only in PAYLOAD.
chk_ok	Input	1	Indicates checksum verification passed. Valid only in CHECKSUM.
chk_fail	Input	1	Indicates checksum verification failed. Valid only in CHECKSUM.
abort	Input	1	Aborts packet reception at any stage, returning FSM to IDLE.
valid_pkt	Output	1	Pulses high when packet successfully received (CHECKSUM to DONE).
error_pkt	Output	1	Pulses high when checksum fails (CHECKSUM to IDLE).
state	Output	3	Encoded FSM state (0-4).

Table 1: Packet Module I/O Interface Signals

2.1 State Encoding

The FSM implements five distinct states, each representing a stage in packet reception:

- **IDLE (0)**: Waiting for packet start.
- **HEADER (1)**: Processing header.

- **PAYLOAD (2)**: Processing payload.
- **CHECKSUM (3)**: Verifying checksum.
- **DONE (4)**: Packet reception complete; automatically returns to IDLE.

2.2 Behavioral description

The packet reception FSM operates according to the following behavioral rules:

1. On reset assertion, the FSM enters the IDLE state and all output signals are cleared.
2. In the IDLE state, assertion of the `start_pkt` signal causes a transition to the HEADER state.
3. In the HEADER state, assertion of the `hdr_done` signal causes a transition to the PAYLOAD state.
4. In the PAYLOAD state, assertion of the `payload_done` signal causes a transition to the CHECKSUM state.
5. In the CHECKSUM state, two conditions are possible:
   - If `chk_ok` is asserted, the FSM transitions to the DONE state and the `valid_pkt` output pulses high.
   - If `chk_fail` is asserted, the FSM returns to the IDLE state and the `error_pkt` output pulses high.
6. In the DONE state, the FSM automatically transitions back to IDLE on the next clock cycle.
7. At any stage in the FSM, assertion of the `abort` signal forces the FSM back to IDLE with all output signals cleared.

2.3 Finite State Machine Graph

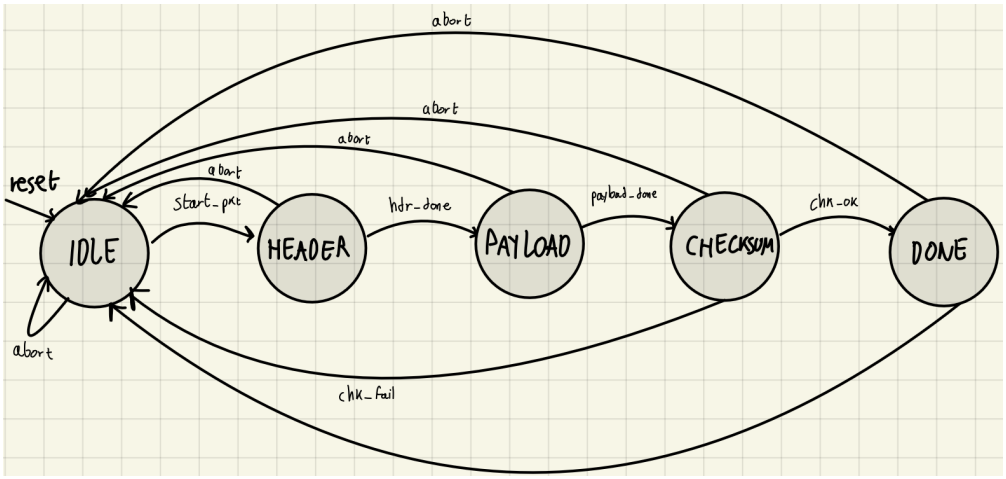

Figure 1: Finite State Machine Graph for Packet Reception

2.4 Formal specification

```
initial assume(reset);

assert property @(posedge clk) if(reset) assert(state == IDLE);
assert property @(posedge clk) if(reset) assume(chk_ok == 1'b0 && chk_fail ==
↪  1'b0);

always @(posedge clk) disable iff (reset) if(state == CHECKSUM) assume(chk_ok !=
↪  chk_fail);

//ALWAYS A STATE

assert property @(posedge clk) disable iff (reset) assert(  state == IDLE
↪  ||
                                                            state == HEADER
                                                            ↪  ||
                                                            state == PAYLOAD
                                                            ↪  ||
                                                            state == CHECKSUM
                                                            ↪  ||
                                                            state == DONE
                                                        );

// CORRECT STATE
assert property @(posedge clk) disable iff (reset) valid_pkt |-> (state == DONE);
assert property @(posedge clk) disable iff (reset) error_pkt |-> (state == IDLE);

// STATE TRANSITIONS
assert property @(posedge clk) disable iff (reset) start_pkt && (state == IDLE)
↪  && !abort |=> (state == HEADER);
assert property @(posedge clk) disable iff (reset) hdr_done && (state == HEADER)
↪  && !abort |=> (state == PAYLOAD);
assert property @(posedge clk) disable iff (reset) payload_done && (state ==
↪  PAYLOAD) && !abort |=> (state == CHECKSUM);
assert property @(posedge clk) disable iff (reset) chk_ok && (state == CHECKSUM)
↪  && !abort |=> (state == DONE);
assert property @(posedge clk) disable iff (reset) chk_fail && (state ==
↪  CHECKSUM) && !abort |=> (state == IDLE);
assert property @(posedge clk) disable iff (reset) (state == DONE) && !abort |=>
↪  (state == IDLE);
assert property @(posedge clk) disable iff (reset) abort |=> (state == IDLE);
```

3 Conclusion

4 Appendix

4.1 Number of hours required

For the completion of this laboratory, it has been needed x hours.

4.2 Appendix A